



# 3D hybrid-parametric inversion of AEM data accelerated by tiling

Lindsey J. Heagy<sup>1</sup>, Joseph Capriotti<sup>2</sup>, Seogi Kang<sup>3</sup>, Dominique Fournier<sup>4</sup>,  
John Weis<sup>1</sup>, Johnathan Kuttai<sup>1</sup>, Devin Cowan<sup>1</sup>, Santiago R. Soler<sup>1</sup>

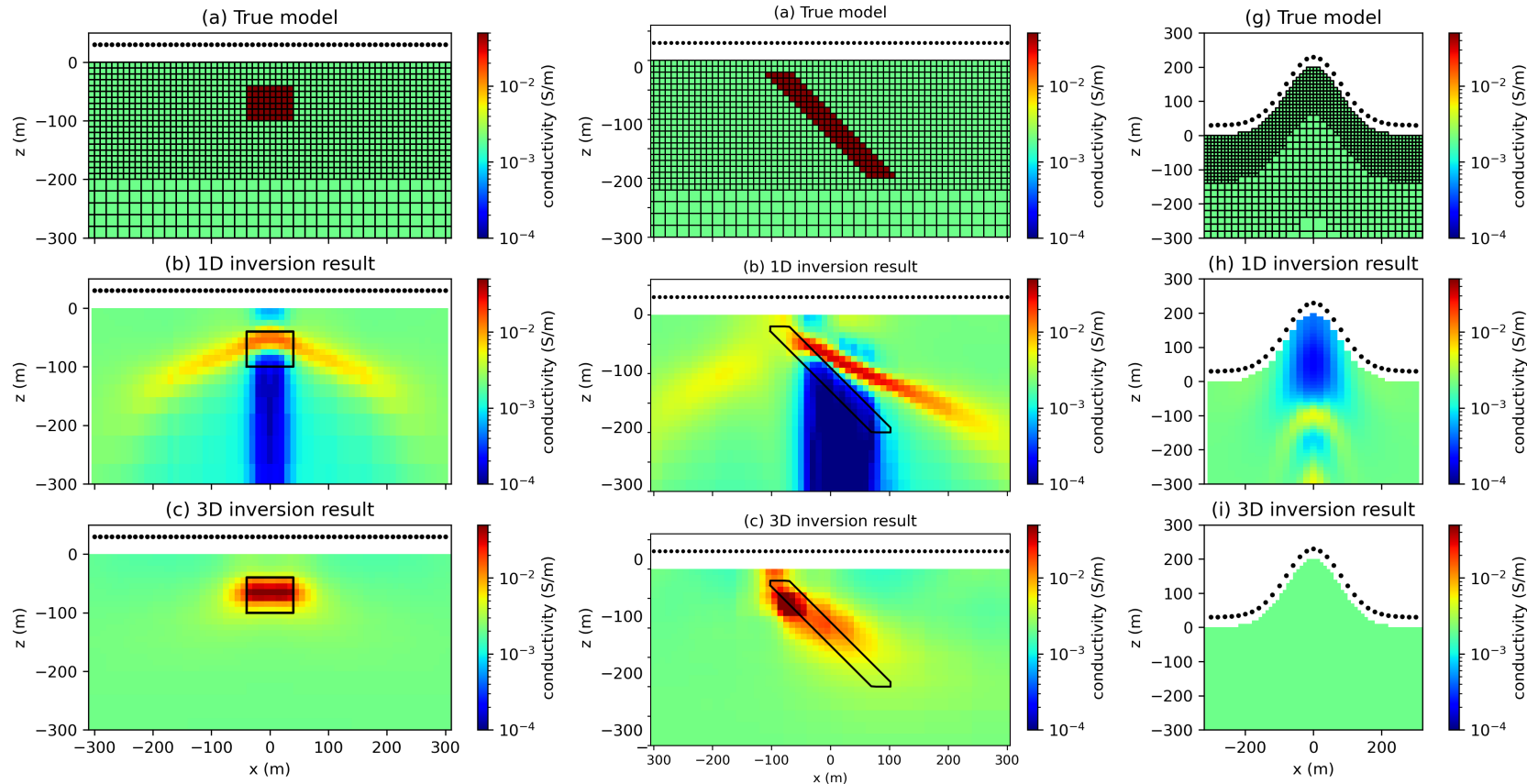
<sup>1</sup> University of British Columbia - Geophysical Inversion Facility

<sup>2</sup> Colorado School of Mines

<sup>3</sup> University of Manitoba

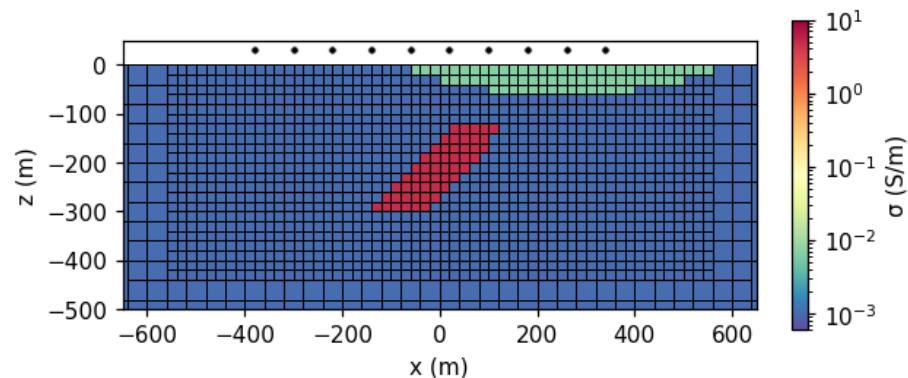
<sup>4</sup> Mira Geoscience

# why 3D?

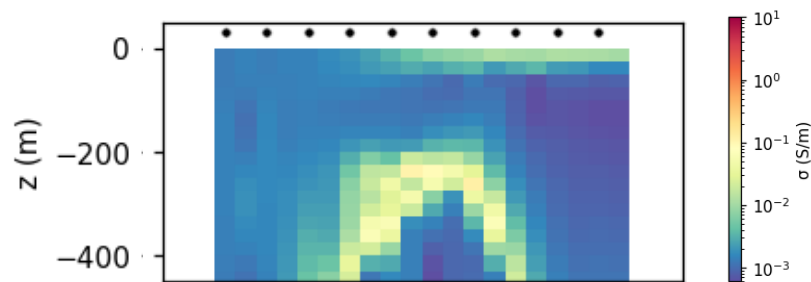


# why 3D?

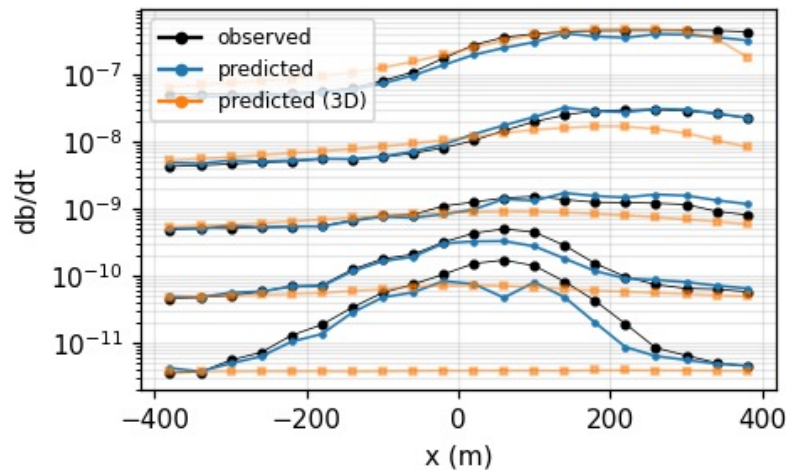
example: conductive target,  
conductive overburden (partial)



Stitched 1D: dip, extent ambiguous

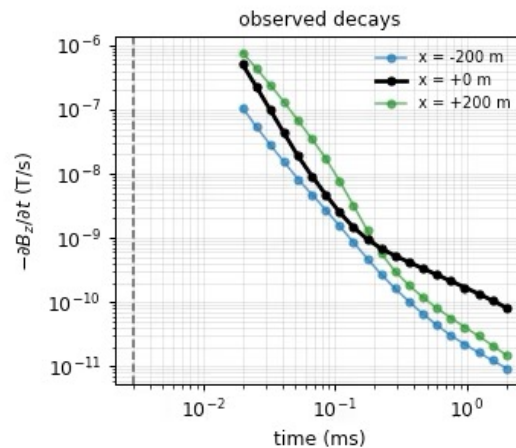
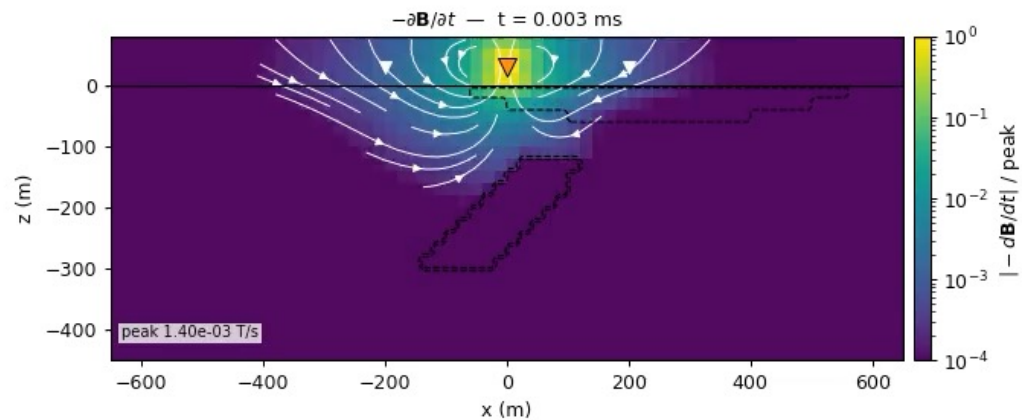
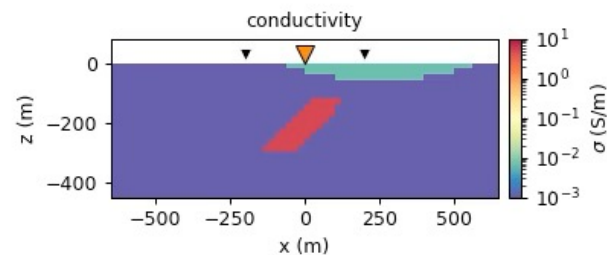
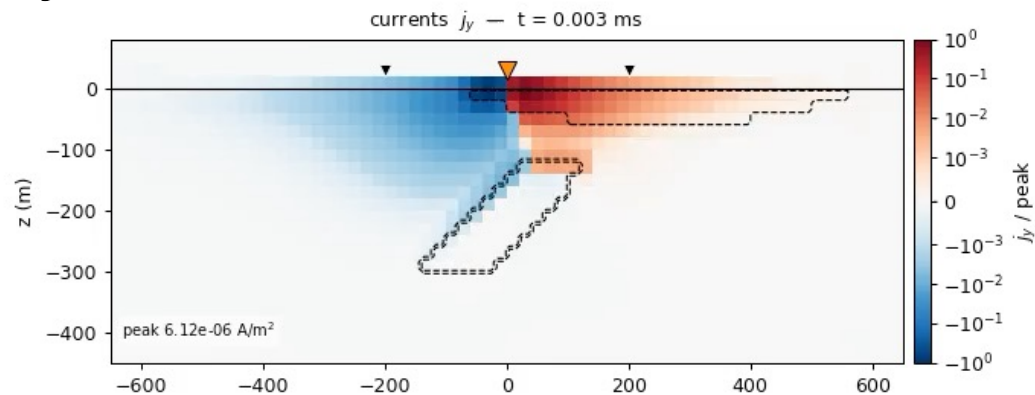


3D forward (orange) shows where 1D  
inversion doesn't fit the data



# why 3D?

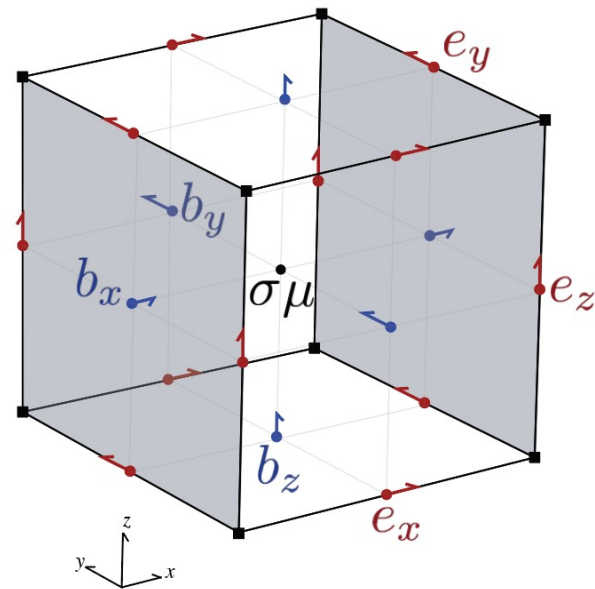
sounding  $x = +0$  m  $t = 0.0030$  ms



# challenges with 3D

Computational cost

Non-uniqueness



# challenges with 3D

## Computational cost.

Solving full 3D Maxwell

$$\nabla \times \mu^{-1} \times \mathbf{e} + \sigma \frac{\partial \mathbf{e}}{\partial t} = -\frac{\partial \mathbf{s}}{\partial t}$$

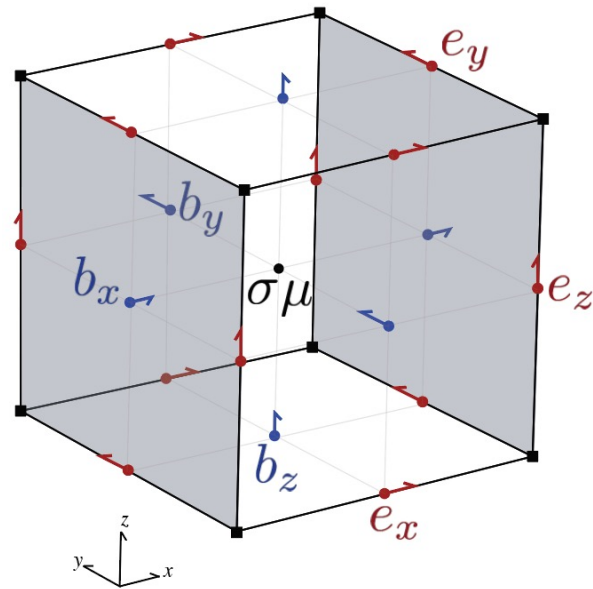
Discretize in space

$$\mathbf{C}^\top \mathbf{M}_{\mu^{-1}}^f \mathbf{C} \mathbf{e} + \mathbf{M}_\sigma^e \frac{\partial \mathbf{e}}{\partial t} = -\frac{\partial \mathbf{s}}{\partial t}$$

Then in time (backward Euler;  $\mathbf{e}^{n+1}$  depends on  $\mathbf{e}^n$ )

- Time step  $\Delta t = t_{n+1} - t_n$

$$\mathbf{C}^\top \mathbf{M}_{\mu^{-1}}^f \mathbf{C} \mathbf{e}^{n+1} + \mathbf{M}_\sigma^e \frac{\mathbf{e}^{n+1} - \mathbf{e}^n}{\Delta t} = -\frac{\mathbf{s}^{n+1} - \mathbf{s}^n}{\Delta t}$$



# challenges with 3D

## Computational cost.

Time stepping problem

$$\underbrace{\left( \mathbf{C}^\top \mathbf{M}_{\mu-1}^f \mathbf{C} + \frac{1}{\Delta t} \mathbf{M}_\sigma^e \right)}_{\mathbf{A}_{n+1}(\sigma, \Delta t)} \underbrace{\mathbf{e}^{n+1}}_{\mathbf{u}_{n+1}} = \underbrace{-\frac{\mathbf{s}^{n+1} - \mathbf{s}^n}{\Delta t}}_{\mathbf{q}_{n+1}} + \underbrace{\frac{1}{\Delta t} \mathbf{M}_\sigma^e}_{\mathbf{B}_n(\sigma, \Delta t)} \underbrace{\mathbf{e}^n}_{\mathbf{u}_n}$$

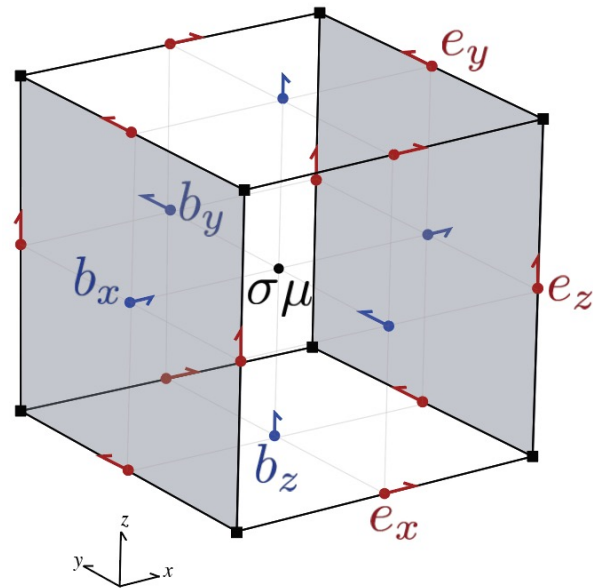
Arrange in a big matrix

$$\begin{pmatrix} \mathbf{A}_0 & & & & & \\ \mathbf{B}_1 & \mathbf{A}_1 & & & & \\ & \mathbf{B}_2 & \mathbf{A}_2 & & & \\ & & \ddots & \ddots & & \\ & & & \mathbf{B}_{n-1} & \mathbf{A}_{n-1} & \\ & & & & \mathbf{B}_n & \mathbf{A}_n \end{pmatrix} \begin{pmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_{n-1} \\ \mathbf{u}_n \end{pmatrix} = \begin{pmatrix} \mathbf{q}_0 \\ \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \\ \mathbf{q}_{n-1} \\ \mathbf{q}_n \end{pmatrix}$$

Solve with forward elimination

- Initial conditions provide  $\mathbf{u}_0$
- To propagate forward, solve

$$\mathbf{A}_{n+1} \mathbf{u}_{n+1} = -\mathbf{B}_n \mathbf{u}_n + \mathbf{q}_{n+1}$$



# challenges with 3D

## Computational cost.

50,000 cells (not a huge mesh...)

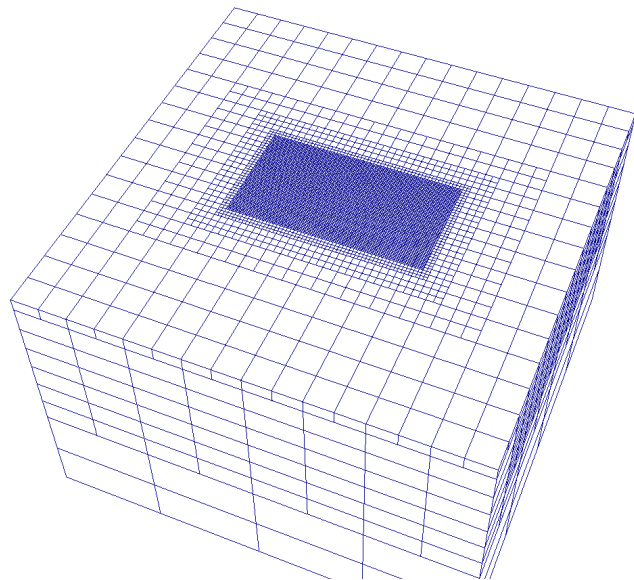
80 time-steps, 4 unique step sizes

For a single sounding:

- 4 factorizations: each ~18 s
- 80 back-substitutions: each ~0.15 s
- total: 90s

Inversion: each iteration may take ~10-20 solves

10 inversion iterations → ~5 hours

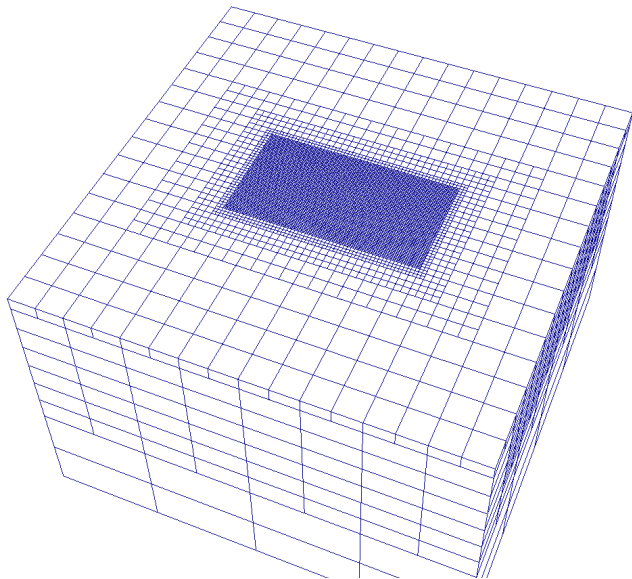


# challenges with 3D

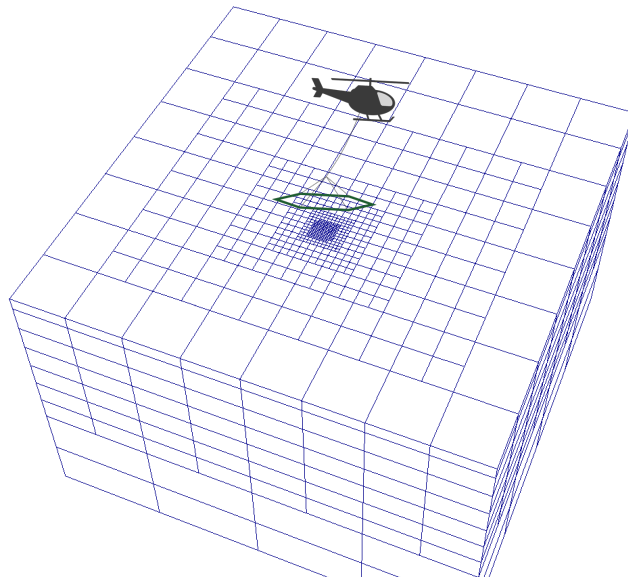
Computational cost.

Solution: tiling (after Yang et al; Cox et al)

Global mesh



Local mesh

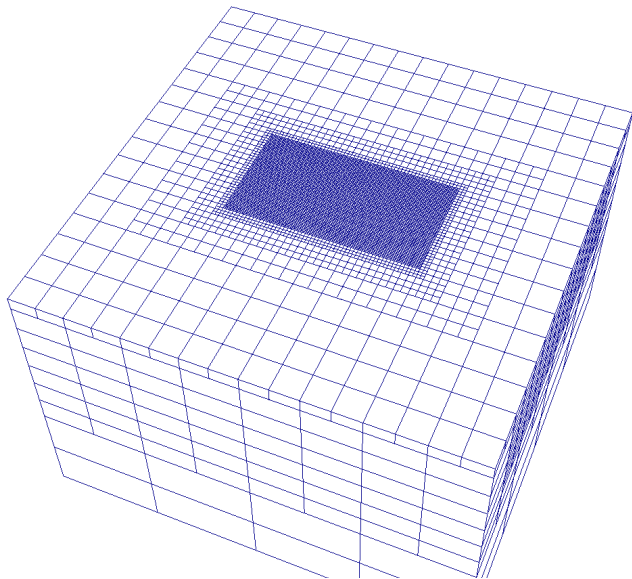


# challenges with 3D

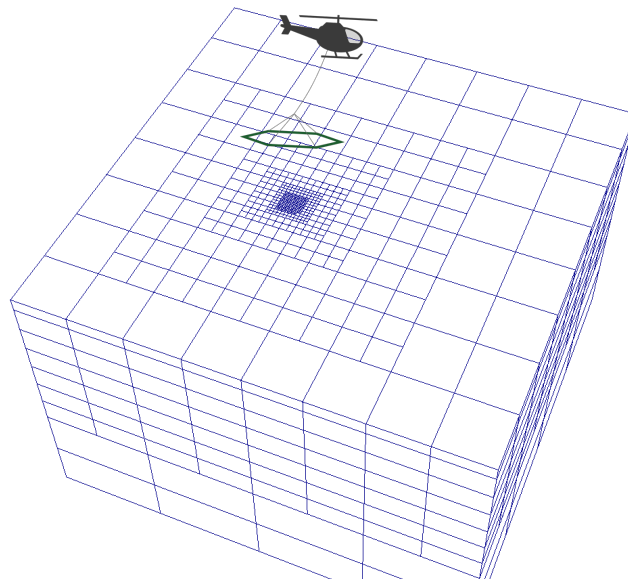
Computational cost.

Solution: tiling (after Yang et al; Cox et al)

Global mesh



Local mesh

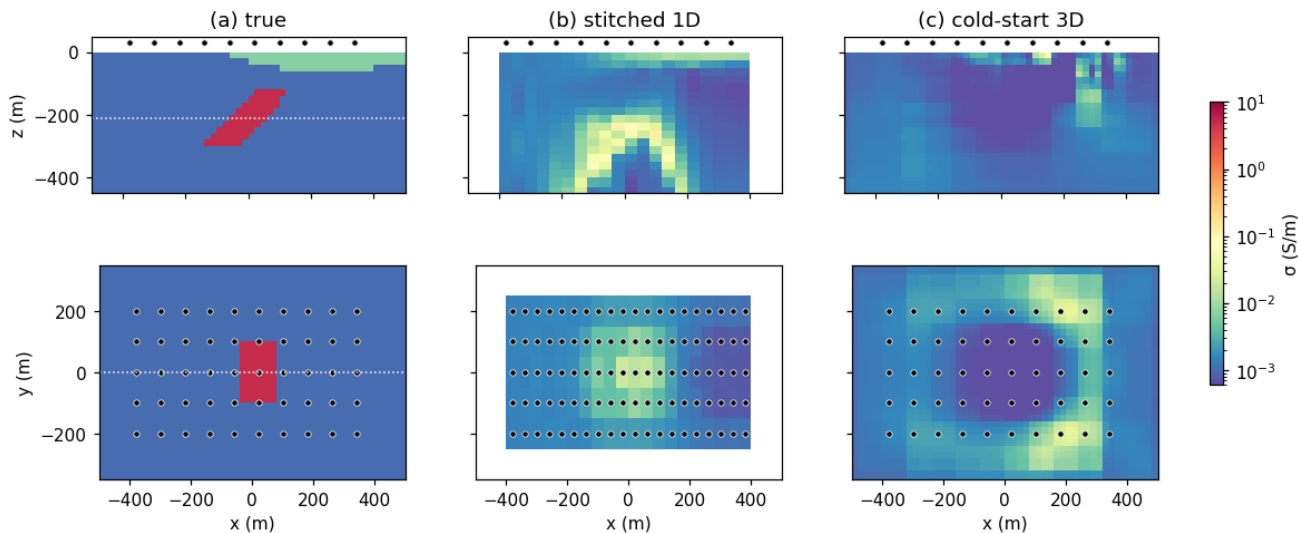


# challenges with 3D

## Nonuniqueness

- Higher dimensional space
- Prone to sensitivity artifacts (see McMillan)

e.g. Cold-started inversion, no sensitivity weighting

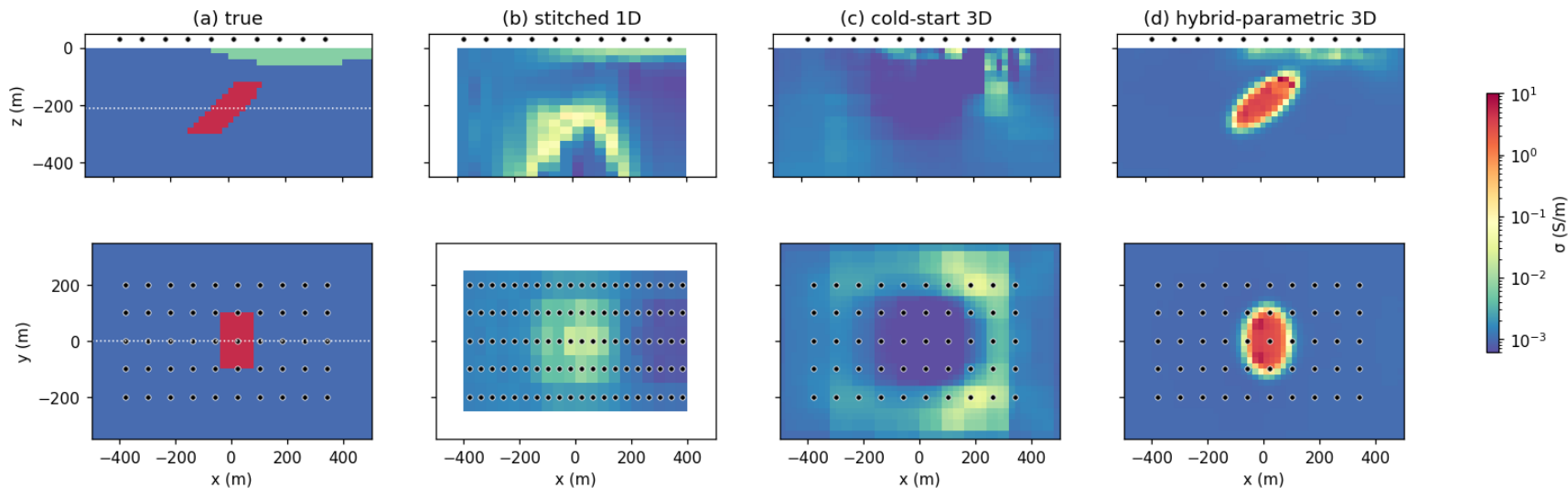


# challenges with 3D

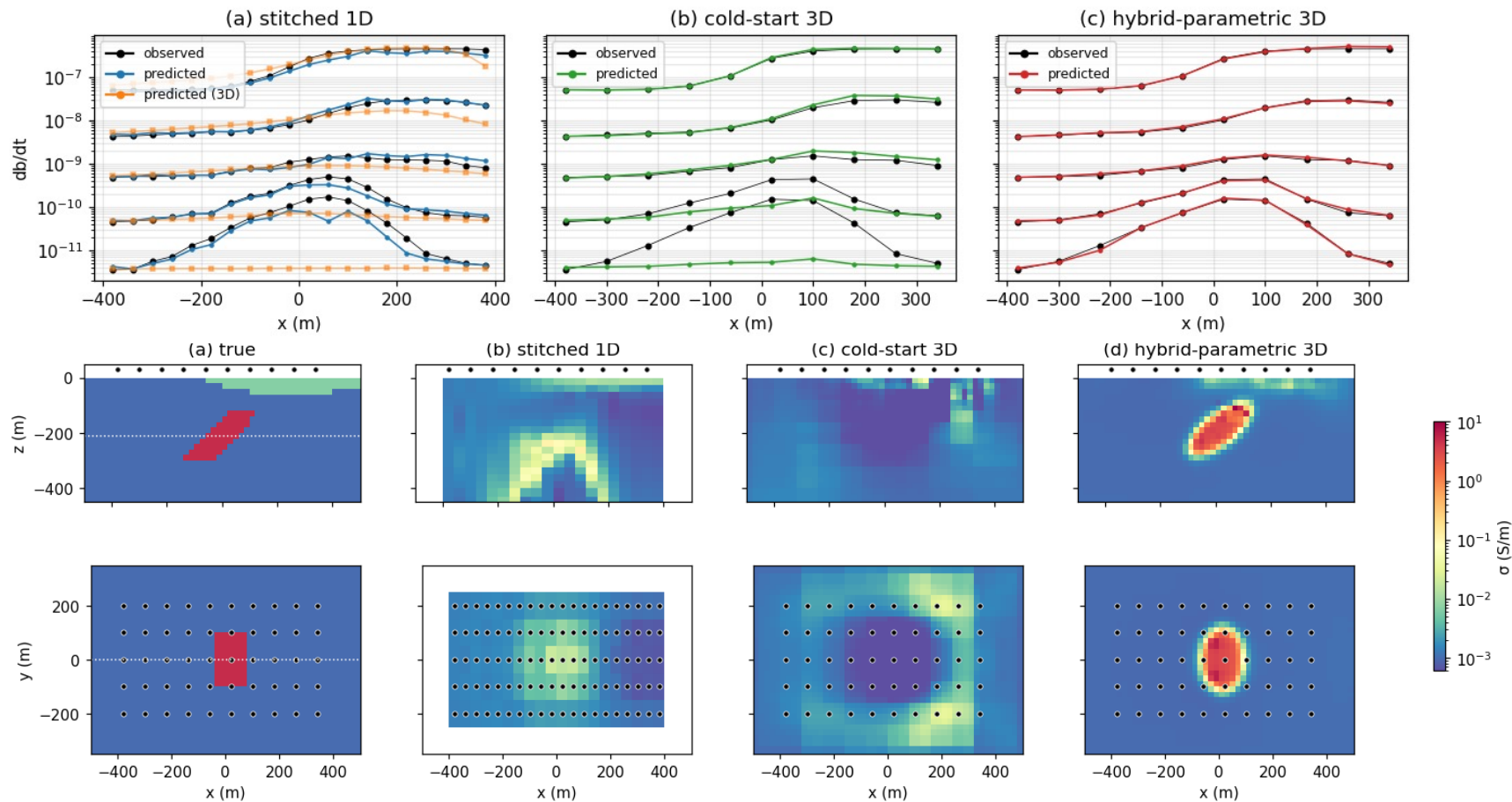
## Nonuniqueness

One solution: parametric / hybrid parametric approaches (after McMillan et al)

- 2 step inversion: invert late times only for best fitting ellipsoid
- Use this as a starting and reference model for a full 3D voxel inversion



# challenges with 3D



# implementation



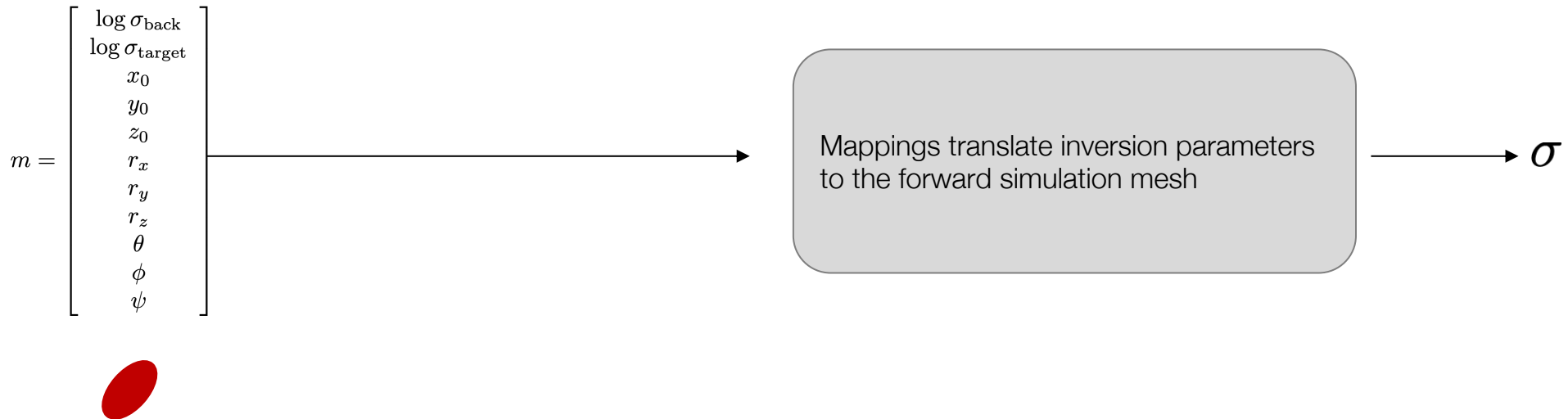
tiling + hybrid parametric inversions in SimPEG

two key components:

- mappings: transformations of model vector, with derivatives
- meta-simulation: for tiling & parallelized simulations

# mappings

Ellipsoid  
parameters



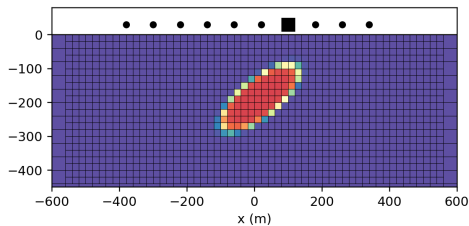
# mappings

Ellipsoid  
parameters

$$m = \begin{bmatrix} \log \sigma_{\text{back}} \\ \log \sigma_{\text{target}} \\ x_0 \\ y_0 \\ z_0 \\ r_x \\ r_y \\ r_z \\ \theta \\ \phi \\ \psi \end{bmatrix}$$



Parametric ellipsoid on  
global mesh (in active cells)



```
maps.ParametricEllipsoid(  
    global_mesh,  
    active_cells,  
    boundary_sharpness=10.0)
```

Parametric ellipsoid

- Level set approach (differentiable)
- Use Pytorch (autodiff!)

$\sigma$

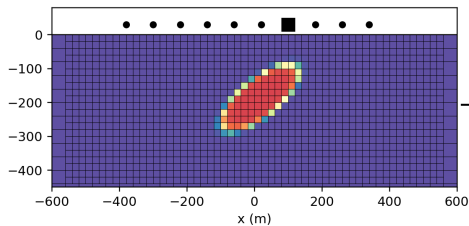
# mappings

Ellipsoid  
parameters

$$m = \begin{bmatrix} \log \sigma_{\text{back}} \\ \log \sigma_{\text{target}} \\ x_0 \\ y_0 \\ z_0 \\ r_x \\ r_y \\ r_z \\ \theta \\ \phi \\ \psi \end{bmatrix}$$

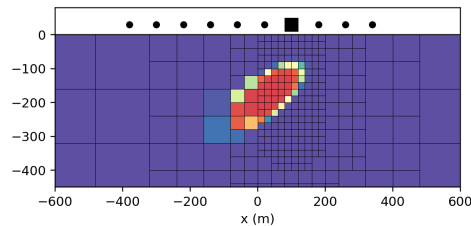


Parametric ellipsoid on  
global mesh (in active cells)



```
maps.ParametricEllipsoid(  
    global_mesh,  
    active_cells,  
    boundary_sharpness=10.0)
```

Volume-averaged model  
on local, tile mesh



```
maps.TileMap(global_mesh,  
    active_cells, local_mesh)
```

$\sigma$

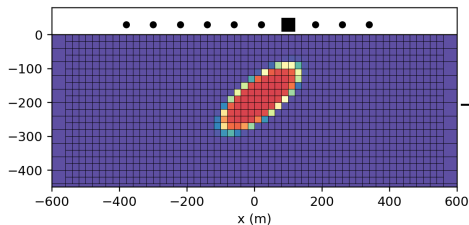
# mappings

Ellipsoid  
parameters

$$m = \begin{bmatrix} \log \sigma_{\text{back}} \\ \log \sigma_{\text{target}} \\ x_0 \\ y_0 \\ z_0 \\ r_x \\ r_y \\ r_z \\ \theta \\ \phi \\ \psi \end{bmatrix}$$

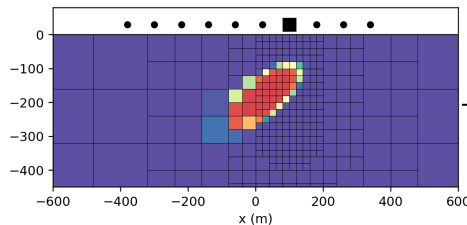


Parametric ellipsoid on  
global mesh (in active cells)



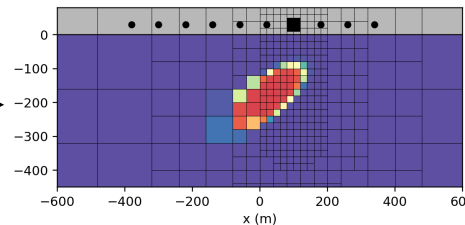
```
maps.ParametricEllipsoid(  
    global_mesh,  
    active_cells,  
    boundary_sharpness=10.0)
```

Volume-averaged model  
on local, tile mesh



```
maps.TileMap(global_mesh,  
    active_cells, local_mesh)
```

Include air cells, take  
exponential



```
maps.InjectActiveCells(  
    local_mesh, active_cells)
```

```
maps.ExpMap(local_mesh)
```

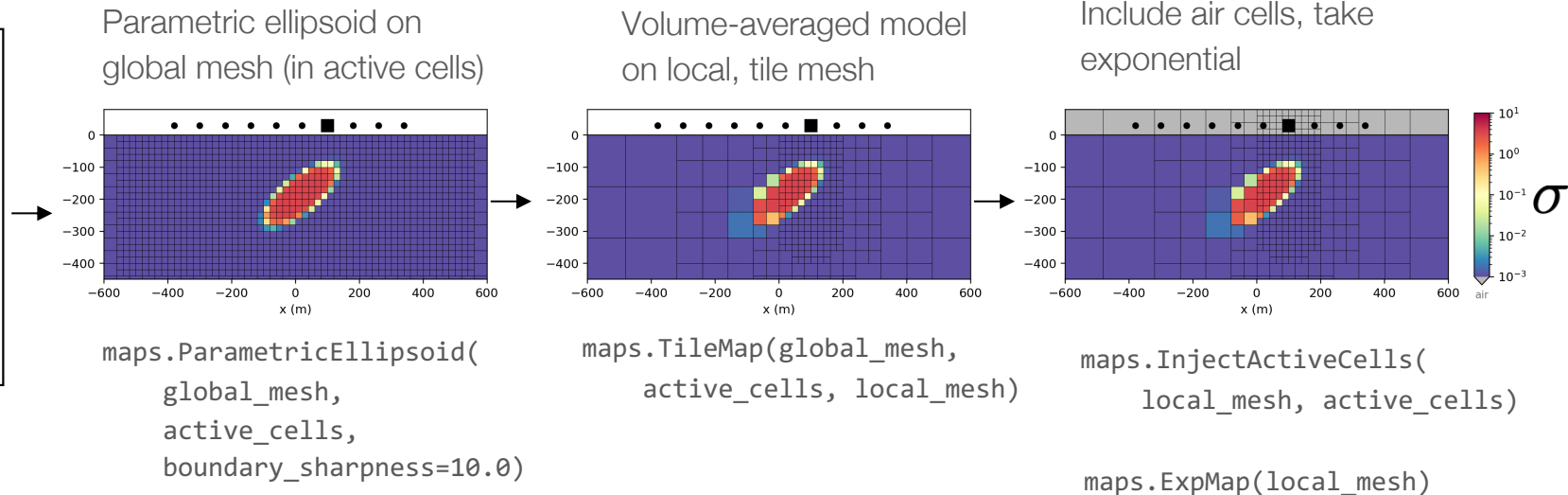
$$\sigma = \mathcal{M}_{\text{exp}} \circ \mathcal{M}_{\text{actv}} \circ \mathcal{M}_{\text{tile}} \circ \mathcal{M}_{\text{ellipsoid}}(m)$$

# mappings

Also need sensitivities!  
Chain rule.

Ellipsoid  
parameters

$$m = \begin{bmatrix} \log \sigma_{\text{back}} \\ \log \sigma_{\text{target}} \\ x_0 \\ y_0 \\ z_0 \\ r_x \\ r_y \\ r_z \\ \theta \\ \phi \\ \psi \end{bmatrix}$$



$$\frac{\partial \sigma}{\partial m} = \frac{\partial}{\partial m} \mathcal{M}_{\text{exp}} \circ \mathcal{M}_{\text{actv}} \circ \mathcal{M}_{\text{tile}} \circ \mathcal{M}_{\text{ellipsoid}}(m)$$

# meta-simulations

book-keeping and parallelization for full forward simulation

$$d^{\text{pred}} = \mathcal{F}[m]$$

two approaches (currently)

- **Multiprocessing:** python's built in multi-threading (single node)
- **Dask:** can parallelize across nodes

```
# source-dependent meshes (tiles by transmitter)
simulations, mappings = [], []
for src in survey.source_list:
```

```
# source-dependent meshes (tiles by transmitter)
simulations, mappings = [], []
for src in survey.source_list:
    local_mesh = build_local_mesh(src, refine_depth=300)
```

```
# source-dependent meshes (tiles by transmitter)
simulations, mappings = [], []
for src in survey.source_list:
    local_mesh = build_local_mesh(src, refine_depth=300)

    # the mapping carries (and volume-averages) the model
    tile = TileMap(global_mesh, active_cells, local_mesh)
    mappings.append(tile)
```

```

# source-dependent meshes (tiles by transmitter)
simulations, mappings = [], []
for src in survey.source_list:
    local_mesh = build_local_mesh(src, refine_depth=300)

    # the mapping carries (and volume-averages) the model
    tile = TileMap(global_mesh, active_cells, local_mesh)
    mappings.append(tile)

# a standard SimPEG TDEM simulation
simulations.append(Simulation3DElectricField(
    local_mesh,
    survey=Survey([src]),
    time_steps=[(3e-6, 20), (1e-5, 20), (3e-5, 20), (1e-4, 20)],
    sigmaMap=expmap * active_cells * ellipsoid_mapping,
))

```

```

# source-dependent meshes (tiles by transmitter)
simulations, mappings = [], []
for src in survey.source_list:
    local_mesh = build_local_mesh(src, refine_depth=300)

    # the mapping carries (and volume-averages) the model
    tile = TileMap(global_mesh, active_cells, local_mesh)
    mappings.append(tile)

    # a standard SimPEG TDEM simulation
    simulations.append(Simulation3DElectricField(
        local_mesh,
        survey=Survey([src]),
        time_steps=[(3e-6, 20), (1e-5, 20), (3e-5, 20), (1e-4, 20)],
        sigmaMap=expmap * active_cells * ellipsoid_mapping,
    ))

# compose: behaves like ONE simulation
sim = MultiprocessingMetaSimulation(simulations, mappings)
dpred = sim.dpred(model) # Jvec / Jtvec too

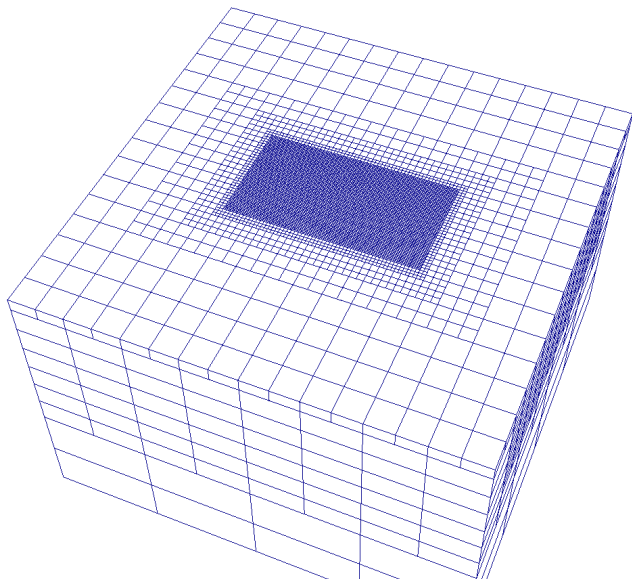
```

# computational savings

## Global mesh

50,000 cells; 90s per simulation

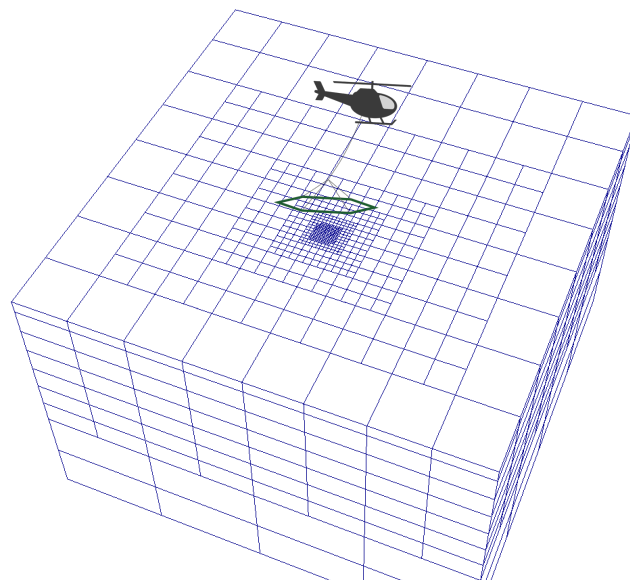
~ 5 hours for 10 inversion iterations



## Tiled approach

5,500 cells per tile; 3s per simulation

~ 10 min for 10 iterations in an inversion



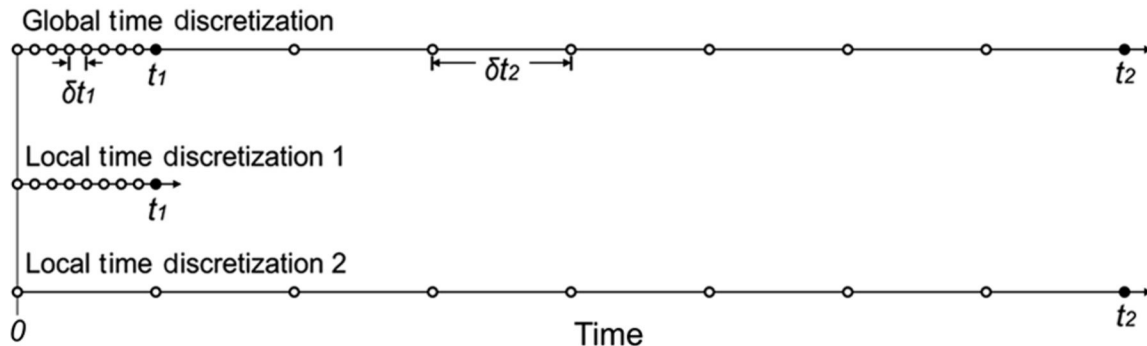
# meta-simulations more broadly

Works with any SimPEG simulation type

- In natural-source / FDEM methods: parallelize over frequency + tiling

Flexible & extensible

- Could be used with different time stepping strategies (Yang & Oldenburg, 2016)



# modular. can be combined with other advanced inversions

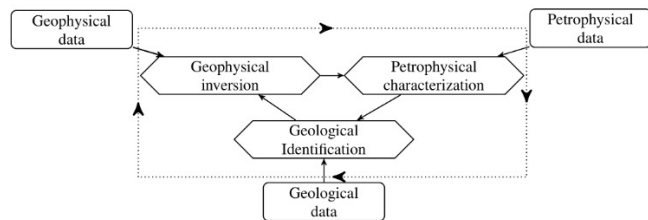


Lp norms (Fournier et al)

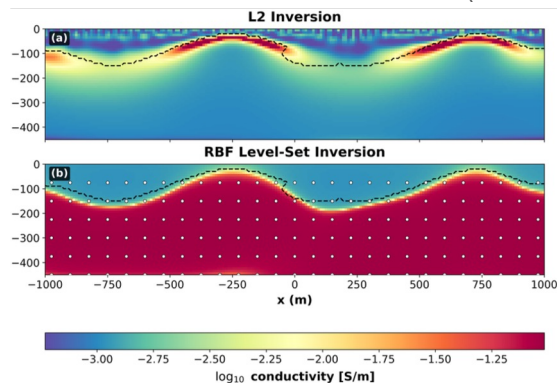
$$\phi_m = \alpha_s \int_V w_s |m - m_{\text{ref}}|^{p_s} dV + \alpha_x \int_V w_x \left| \frac{d(m - m_{\text{ref}})}{dx} \right|^{p_x} dV$$



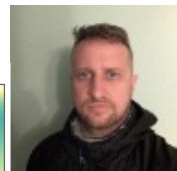
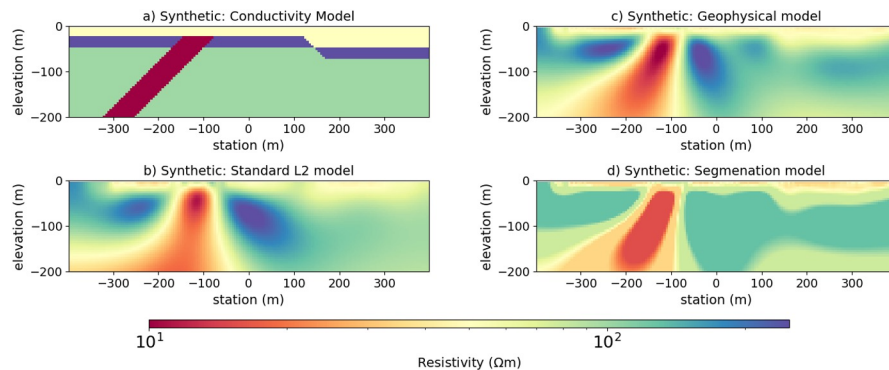
Petrophysically and geologically guided inversion (Astic et al)



Parametric inversions with RBFs (Pokar et al)



Segmentation-guided inversion (Kuttai et al)



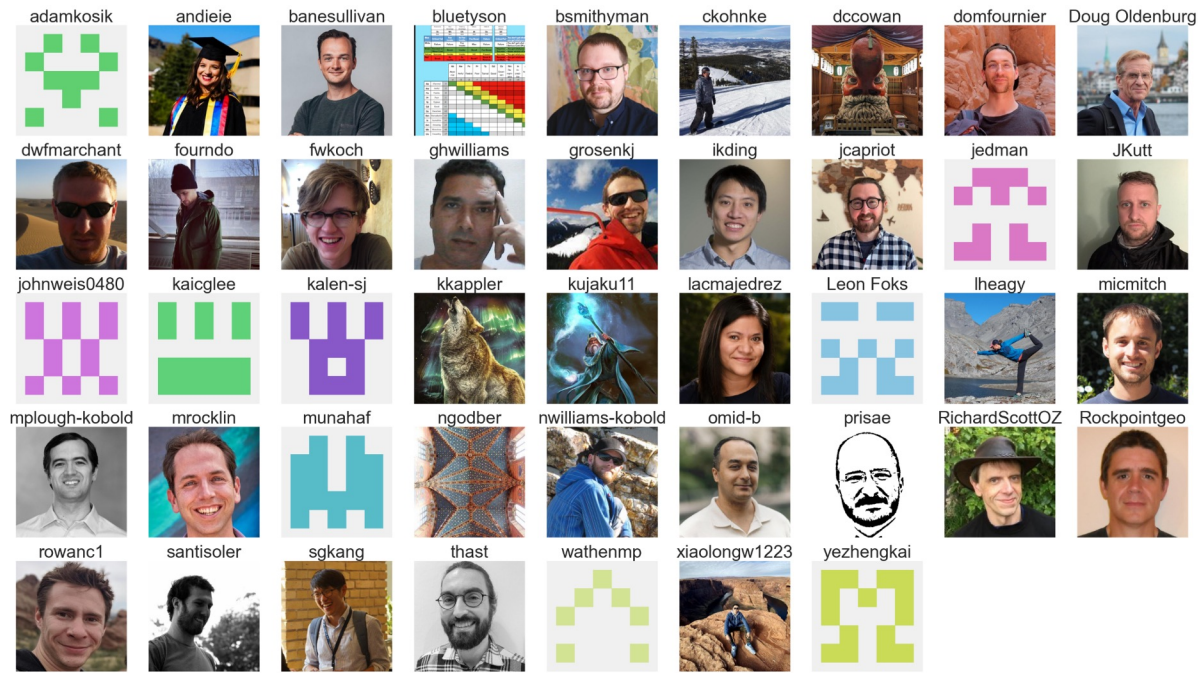
# thank you!



[simpeg.xyz](https://simpeg.xyz)



[lheagy@eoas.ubc.ca](mailto:lheagy@eoas.ubc.ca)



3DEM-8 in Rio de Janeiro, Apr 6 - 8, 2027

# **HOHMANN WANNAMAKER TRUST**

for Electromagnetic Geophysics

<https://www.hohmannwannamakertrust.org>